REFINED QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

Table of Content

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 – Team Fiveo'clock
Team Repo URL	https://github.com/tzee27/AdsGenerator
Project Board URL	https://docs.google.com/spreadsheets/d/1C7fycDPc2OrRZ6tVFSVmd1b9Rq93HLxBvUddAwK-ve4/edit?usp=sharing
Live Deployment URL	https://ads-generator-five.vercel.app

Objective:

The primary of this Refined Quality Assurance Testing Documentation (QATD) is to provide concrete, evidence-backed validation that AdKedai's core pipeline: CSV inventory upload, GLM-powered advertising strategy recommendation, and AI content generation, performs reliably and accurately under real conditions. This document captures the execution result of all test cases drafted in the Preliminary Round QATD, presents automated test coverage evidence from the CI/CD pipeline, logs all defects encountered with root cause analysis and fix traceability with Git commit hashes, documents known deferred issues transparently, and defines the operational boundaries of the AdKedai prototype ahead of the live demo. All GLM output incidents encountered during testing are logged and resolved or formally deferred with justification.

FINAL ROUND (Execution, RCA & Hard Evidence)

1. Test Execution & Evidence (Manual)

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
TC-01	**Happy Case (Entire Flow):** Advertisement Generator	Verify that users can complete the full workflow from inventory upload to ad content export.	System successfully generates recommendations and exportable ad content.	Workflow completed successfully without errors.	Passed	
TC-02	**Specific Case (Negative):** Invalid CSV Input	Verify that the system rejects invalid inventory files.	The system rejects file and displays a validation error message.	The system rejects the files and shows the invalid part that appear in CSV	Passed	
TC-03	**NFR (Performance):** Large Inventory	Verify system performance when processing large inventory uploads.	System processes inventory within 45 seconds.	Inventory processed before 45 seconds	Passed	
TC-AI-01	**AI Output Validation:** GLM Image Generation	Validated image generation for Farm Fresh milk. Checked for	The generated image should show 1L	The image correctly shows the 1L bottle with dynamic milk	Passed	

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
		product accuracy, promotional badge (30% OFF), and Wesak Day theme.	bottle with dynamic milk splashes and green leaves. The "Flash Sale" badge should be clear and visible.	splashes and green leaves. The "Flash Sale" badge is clear, and the lotus lanterns are visible in the blurred background.		

2. Automated Testing Pipeline

2.1. Unit Testing (Core Logic & Functions)

Unit testing is primarily focused on Isolation of the individual components or functions to ensure that these functionalities work perfectly on their own without the need of any external dependencies. Mock your databases or APIs here.

- **Test Runner Used:** [Pytest (Python)]
- **Targeted Components:** [analyse_csv(), gather_live_context(), compute_financial_projection(), decide_strategy(), generate_image()]
- **File(s) Covered:**

  [https://github.com/tzee27/AdsGenerator/blob/main/backend/tests/test_risk_analyser.py,
  https://github.com/tzee27/AdsGenerator/blob/main/backend/tests/test_context_gatherer.py,
  https://github.com/tzee27/AdsGenerator/blob/main/backend/tests/test_explanation_generator.py,
  https://github.com/tzee27/AdsGenerator/blob/main/backend/tests/test_strategy_decider.py,
  https://github.com/tzee27/AdsGenerator/blob/main/backend/tests/test_content_generator.py,
  https://github.com/tzee27/AdsGenerator/blob/main/backend/tests/test_glm_image_client.py]

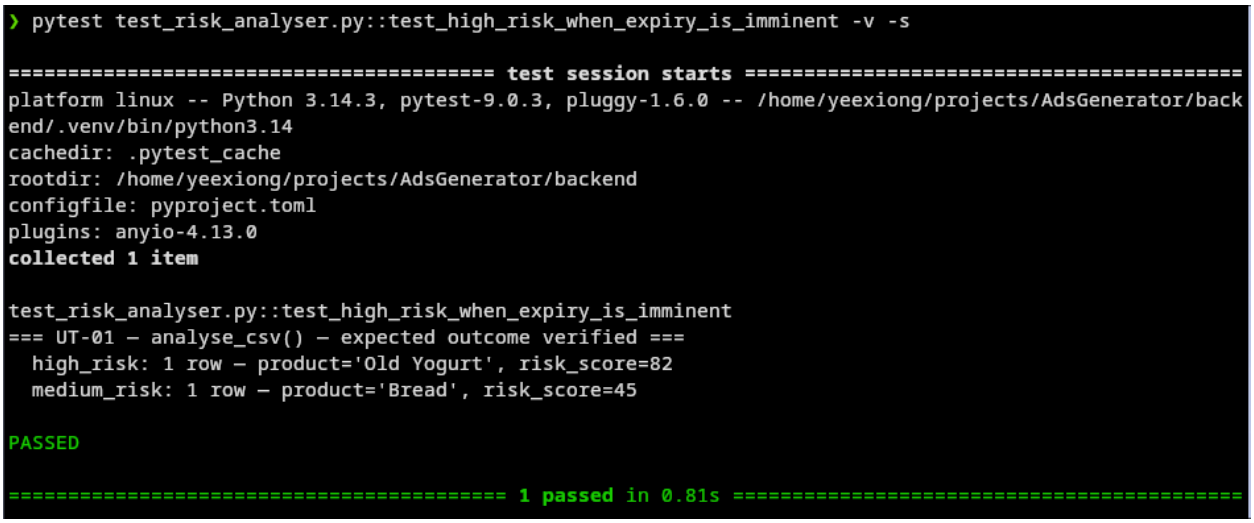

Figure 2.1.1: Unit Test Result (CSV File Analysis)

Unit Test ID	Function/ Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-01	analyse_csv()	Two-row CSV: one product high risk (imminent expiry + age + exposure), one medium	Exactly one row in high_risk (“Old Yogurt”, score 82); one in medium_risk (“Bread”, score 45)	test_risk_analyser.py::test_high_risk_when_expiry_is_imminent === UT-01 — analyse_csv() — expected outcome verified === high_risk: 1 row — product='Old Yogurt', risk_score=82 medium_risk: 1 row — product='Bread', risk_score=45	**Passed**

Email: umhackathon@um.edu.my

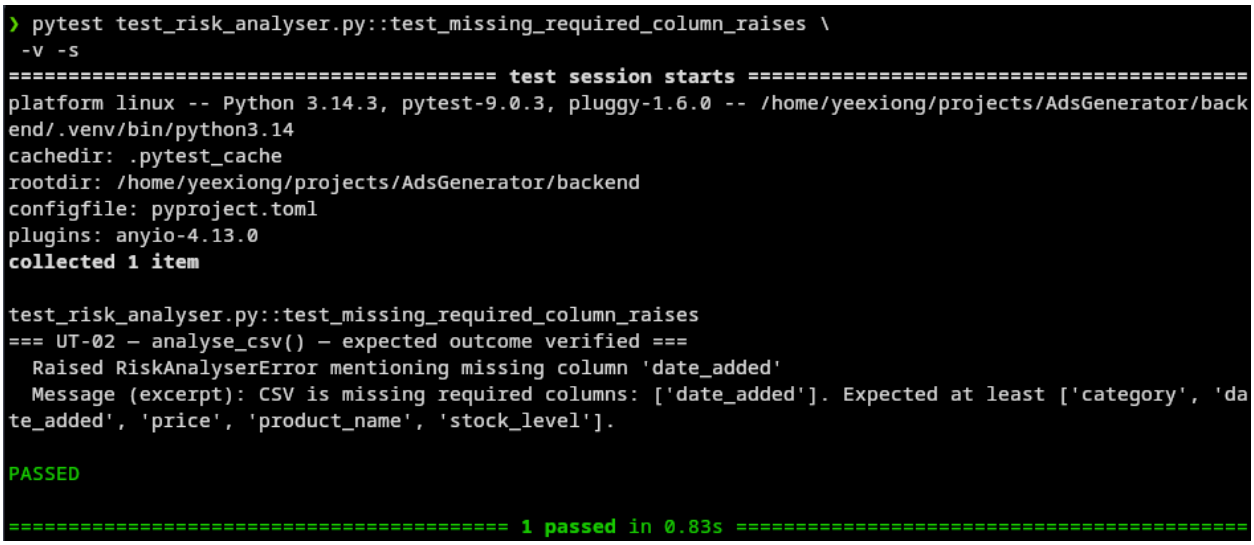

Figure 2.1.2: Unit Test Result (CSV Missing Required Columns)

Unit Test ID	Function/Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-02	analyse_csv()	CSV missing required column	Raises RiskAnalyserError mentioning missing column	test_risk_analyser.py::test_missing_required_column_raises === UT-02 — analyse_csv() — expected outcome verified === Raised RiskAnalyserError mentioning missing column 'date_added' Messagez (excerpt): CSV is missing required columns: ['date_added']. Expected at least ['category', 'date_added', 'price', 'product_name', 'stock_level'].	**Passed**

Email: umhackathon@um.edu.my

```
❯ pytest test_context_gatherer.py::test_gather_context_happy_path \
-v -s
========================================= test session starts ==========================================
platform linux -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- /home/yeexiong/projects/AdsGenerator/back
end/.venv/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/yeexiong/projects/AdsGenerator/backend
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 1 item

test_context_gatherer.py::test_gather_context_happy_path
=== UT-03 — gather_live_context() — expected outcome verified ===
  area='Kuala Lumpur', seasonal_opportunity matches JSON fixture
  enable_web_search (GLM kwargs)=True
  user prompt contains date, area, and product name

PASSED
```

Figure 2.1.3: Unit Test Result (Context Parsing)

Unit Test ID	Function/Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-03	gather_live_context()	Injected glm_fn returns normal JSON context	Parsed LiveContextResponse; area and seasonal copy match; web search flag passed through	test_context_gatherer.py::test_gather_context_happy_path === UT-03 — gather_live_context() — expected outcome verified === area='Kuala Lumpur', seasonal_opportunity matches JSON fixture enable_web_search (GLM kwargs)=True user prompt contains date, area, and product name	**Passed**

Email: umhackathon@um.edu.my

Figure 2.1.4: Unit Test Result (Platform Insights)

Unit Test ID	Function/ Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-04	gather_live_context()	LLM returns platform_insights as list of objects	Normalized dict keys (tiktok, facebook, …)	test_context_gatherer.py::test_gather_context_handles_list_shaped_platform_insights === UT-04 — gather_live_context() — expected outcome verified === platform_insights list from LLM normalized to dict: {'tiktok': 'CPM RM 8', 'facebook': 'CPM RM 5'}	Passed

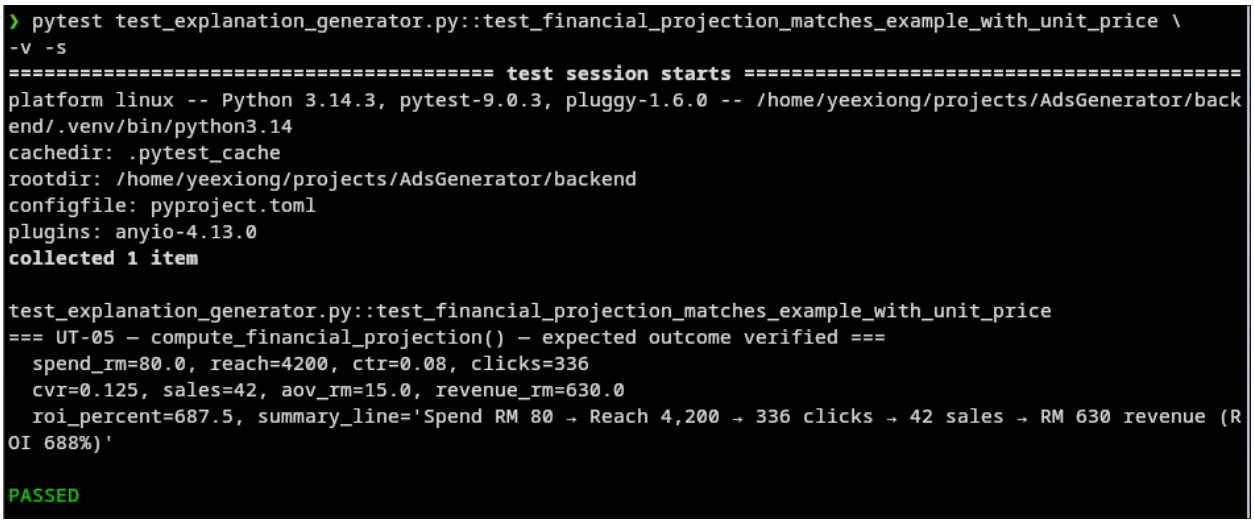

Figure 2.1.5: Unit Test Result (Financial Projection)

Unit Test ID	Function/Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-05	compute_financial_projection()	Fixed budget / reach / assumptions per pipeline spec	Matches documented funnel (e.g. clicks, sales, revenue, ROI %)	test_explanation_generator.py::test_financial_projection_matches_example_with_unit_price === UT-05 — compute_financial_projection() — expected outcome verified === spend_rm=80.0, reach=4200, ctr=0.08, clicks=336 cvr=0.125, sales=42, aov_rm=15.0, revenue_rm=630.0 roi_percent=687.5, summary_line='Spend RM 80 → Reach 4,200 → 336 clicks → 42 sales → RM 630 revenue (ROI 688%)'	Passed

```
❯ pytest test_strategy_decider.py::test_drops_duplicate_product_platform_pairs \
-v -s
========================================= test session starts ==========================================
platform linux -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- /home/yeexiong/projects/AdsGenerator/back
end/.venv/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/yeexiong/projects/AdsGenerator/backend
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 1 item

test_strategy_decider.py::test_drops_duplicate_product_platform_pairs
=== UT-06 — decide_strategy() — expected outcome verified ===
  Duplicate (product, platform) dropped; still 3 strategies with 3 distinct pairs.
  pairs=[('Artisan Sourdough Bread', 'TikTok'), ('Vitamin C Serum 30ml', 'Instagram'), ('Wireless Earbud
s Pro', 'TikTok')]

PASSED
```

Figure 2.1.6: Unit Test Result (Duplicate Product)

Unit Test ID	Function/ Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-06	decide_strategy()	Duplicate (product, platform) in model output	Duplicate does not affect in producing 3 distinct strategies	test_strategy_decider.py::test_drops_duplicate_product_platform_pairs === UT-06 — decide_strategy() — expected outcome verified === Duplicate (product, platform) dropped; still 3 strategies with 3 distinct pairs. pairs=[('Artisan Sourdough Bread', 'TikTok'), ('Vitamin C Serum 30ml', 'Instagram'), ('Wireless Earbuds Pro', 'TikTok')]	**Passed**

```
❯ pytest test_content_generator.py::test_generate_content_happy_path \
-v -s
========================================= test session starts ==========================================
platform linux -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- /home/yeexiong/projects/AdsGenerator/back
end/.venv/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/yeexiong/projects/AdsGenerator/backend
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 1 item

test_content_generator.py::test_generate_content_happy_path
=== UT-07 — generate_content() — expected outcome verified ===
  variants=3, first_headline="Only 12 left — and they're going fast"
  image url='https://fake.url/image.png', mime='image/png'
  enable_web_search=False
  image_fn received prompt prefix + platform/format_hint for aspect ratio

PASSED
```

Figure 2.1.7: Unit Test Result (Three Content Variant)

Unit Test ID	Function/ Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-07	generate_content()	Injected glm_fn returns three content_variants plus image_prompt; image_fn returns a fake PNG result.	Three variants parsed; first headline matches fixture; hashtags start with #; image URL and MIME type set; image_prompt forwarded to image_fn with platform /	test_content_generator.py::test_generate_content_happy_path === UT-07 — generate_content() — expected outcome verified === variants=3, first_headline="Only 12 left — and they're going fast" image url='https://fake.url/image.png', mime='image/png' enable_web_search=False image_fn received prompt prefix + platform/format_hint for aspect ratio	Passed

Email: umhackathon@um.edu.my

			format_hint; GLM called with enable_web_search False; user message includes strategy, product, area, date.		

```
❯ pytest test_glm_image_client.py::test_generate_image_raises_when_key_missing \
  -v -s
========================================= test session starts ==========================================
platform linux -- Python 3.14.3, pytest-9.0.3, pluggy-1.6.0 -- /home/yeexiong/projects/AdsGenerator/back
end/.venv/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/yeexiong/projects/AdsGenerator/backend
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 1 item

test_glm_image_client.py::test_generate_image_raises_when_key_missing
=== UT-08 — generate_image() — expected outcome verified ===
  With empty API key, generate_image() raises GLMImageNotConfiguredError

PASSED
```

Figure 2.1.8: Unit Test Result (Missing API Key)

Unit Test ID	Function/ Component Tested	Test Scenario	Expected Outcome	Actual Result	Status

UT-08	generate_image() (client)	Missing API key	Raises GLMImageNotConfiguredError	test_glm_image_client.py::test_generate_image_raises_when_key_missing === UT-08 — generate_image() — expected outcome verified === With empty API key, generate_image() raises GLMImageNotConfiguredError	Passed

Email: umhackathon@um.edu.my

2.2. Integration & API Testing (System Modules)

- **Test Tool Used:** Postman
- **Targeted Integrations:** FastAPI v1 HTTP API (/api/v1/...) wiring the ads pipeline pieces to deterministic Python services and Z.AI (GLM chat with optional web search, plus GLM-Image for creatives), inventory CSV upload and risk analyser, live context (LLM + web_search_used), strategy options from LLM, ad copy variants (LLM) and image (GLM-Image URL/base64), deterministic financial projection plus LLM platform_choice / risk_vs_reward.
- **File(s) Covered:Risk:**
  https://github.com/tzee27/AdsGenerator/blob/main/backend/app/api/v1/endpoints/risk.py
  https://github.com/tzee27/AdsGenerator/blob/main/backend/app/api/v1/endpoints/context.py
  https://github.com/tzee27/AdsGenerator/blob/main/backend/app/api/v1/endpoints/strategy.py
  https://github.com/tzee27/AdsGenerator/blob/main/backend/app/api/v1/endpoints/content.py
  https://github.com/tzee27/AdsGenerator/blob/main/backend/app/api/v1/endpoints/explanation.py

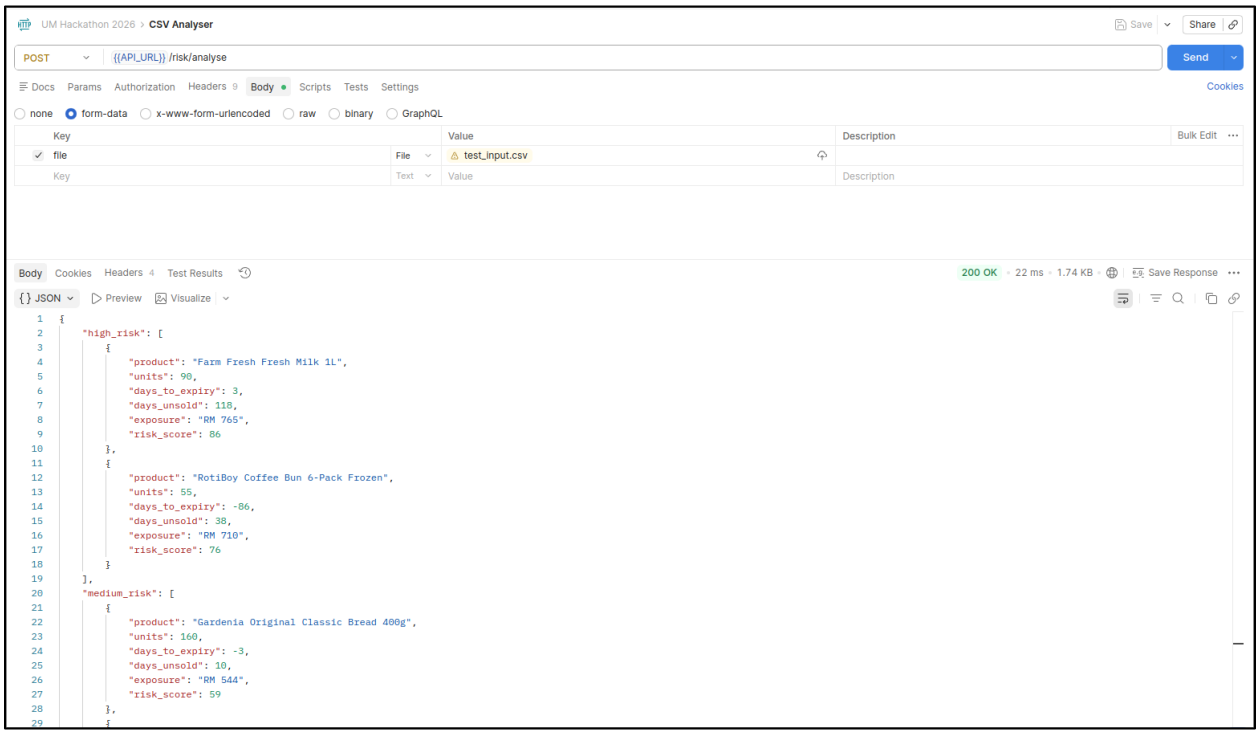

Figure 2.2.1: Postman Result (Risk Analysis)

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status

IT-01	POST /api/v1/risk/analyse → risk analyser (CSV → buckets)	Upload a valid UTF-8 CSV with the expected columns via multipart field file.	Should return HTTP 200. Body matches risk schema (high_risk / medium_risk / low_risk).	{
				"high_risk": [
				{
				"product": "Farm Fresh Fresh Milk 1L",
				"units": 90,
				"days_to_expiry": 3,
				"days_unsold": 118,
				"exposure": "RM 765",
				"risk_score": 86
				},
				{
				"product": "RotiBoy Coffee Bun 6-Pack Frozen",
				"units": 55,
				"days_to_expiry": -86,
				"days_unsold": 38,
				"exposure": "RM 710",
				"risk_score": 76
				}
				],
				"medium_risk": [
				{
				"product": "Gardenia Original Classic Bread 400g",
				"units": 160,
				"days_to_expiry": -3,
				"days_unsold": 10,
				"exposure": "RM 544",
				"risk_score": 59
				},
				{
					Passed

Email: umhackathon@um.edu.my

				"product": "Jasmine Super 5 Rice 5kg", "units": 70, "days_to_expiry": 608, "days_unsold": 60, "exposure": "RM 2,373", "risk_score": 47 }, { "product": "Maggi 2-Minute Curry Noodles 5-Pack", "units": 150, "days_to_expiry": 166, "days_unsold": 102, "exposure": "RM 1,335", "risk_score": 41 }, { "product": "Ayam Brand Sardines in Tomato Sauce 425g", "units": 110, "days_to_expiry": 245, "days_unsold": 64, "exposure": "RM 1,012", "risk_score": 41 }, { "product": "Kimball Spaghetti Pasta 400g", "units": 210, "days_to_expiry": 263, "days_unsold": 92, "exposure": "RM 966", "risk_score": 41 },	

```
      {
        "product": "Adabi Chicken Curry Powder 250g",
        "units": 130,
        "days_to_expiry": 522,
        "days_unsold": 77,
        "exposure": "RM 832",
        "risk_score": 41
      },
      {
        "product": "Quaker Instant Oatmeal 800g",
        "units": 95,
        "days_to_expiry": 223,
        "days_unsold": 69,
        "exposure": "RM 1,378",
        "risk_score": 41
      },
      {
        "product": "Julie’s Peanut Butter Sandwich Biscuits 360g",
        "units": 190,
        "days_to_expiry": 232,
        "days_unsold": 104,
        "exposure": "RM 1,368",
        "risk_score": 41
      }
    ],
    "low_risk": [
      {
        "product": "Dutch Lady Full Cream UHT Milk 1L",
        "units": 140,
        "days_to_expiry": 26,
        "days_unsold": 0,
```


Email: umhackathon@um.edu.my

				"exposure": "RM 1,092",
				"risk_score": 26
				},
				{
				"product": "Nestle Milo Activ-Go 1kg Refill Pack",
				"units": 500,
				"days_to_expiry": 120,
				"days_unsold": 0,
				"exposure": "RM 9,950",
				"risk_score": 20
				}
				]
				}	

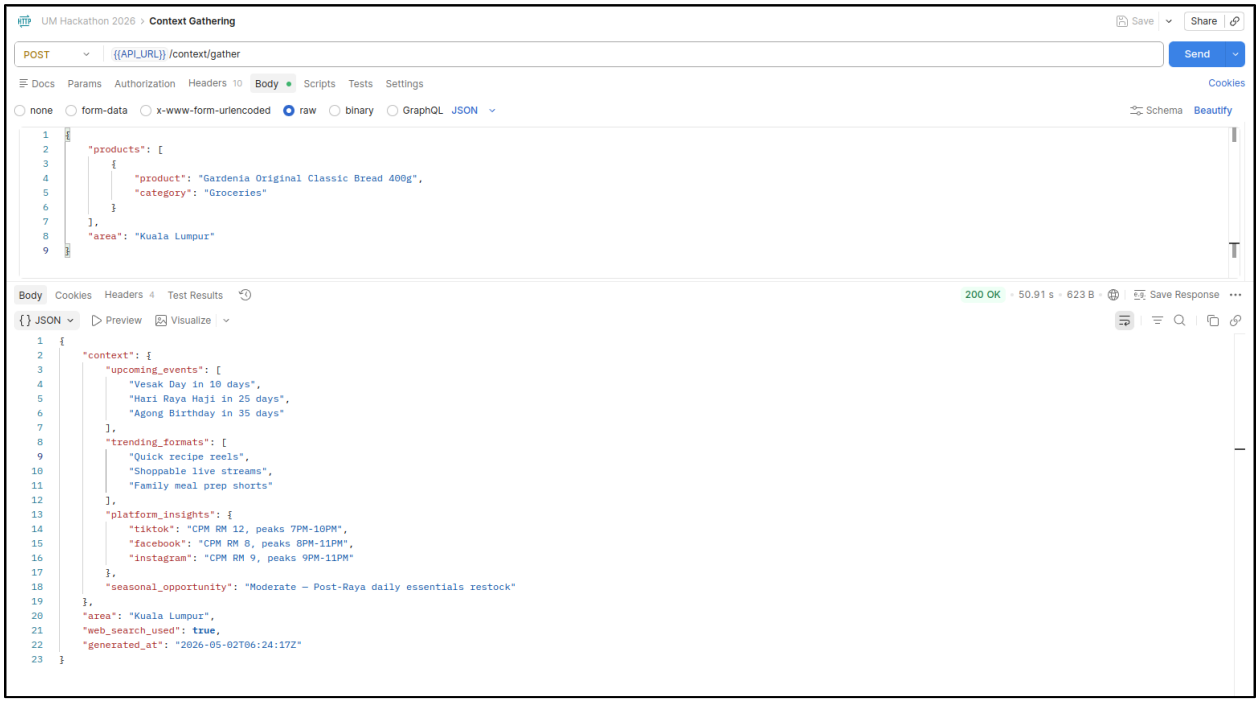

Figure 2.2.2: Postman Result (Context Gathering)

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-02	POST /api/v1/context/gather → Z.A.I GLM (web context)	Send valid JSON with products array and optional area; ZAI_API_KEY set on server.	Should return HTTP 200 with context, area, web_search_used.	{ "context": { "upcoming_events": ["Vesak Day in 10 days", "Hari Raya Haji in 25 days", "Agong Birthday in 35 days"], "trending_formats": ["Quick recipe reels", "Shoppable live streams", "Family meal prep shorts"], "platform_insights": { "tiktok": "CPM RM 12, peaks 7PM-10PM", "facebook": "CPM RM 8, peaks 8PM-11PM", "instagram": "CPM RM 9, peaks 9PM-11PM" },	Passed

Email: umhackathon@um.edu.my

				"seasonal_opportunity": "Moderate — Post-Raya daily essentials restock" }, "area": "Kuala Lumpur", "web_search_used": true, "generated_at": "2026-05-02T06:24:17Z" }	

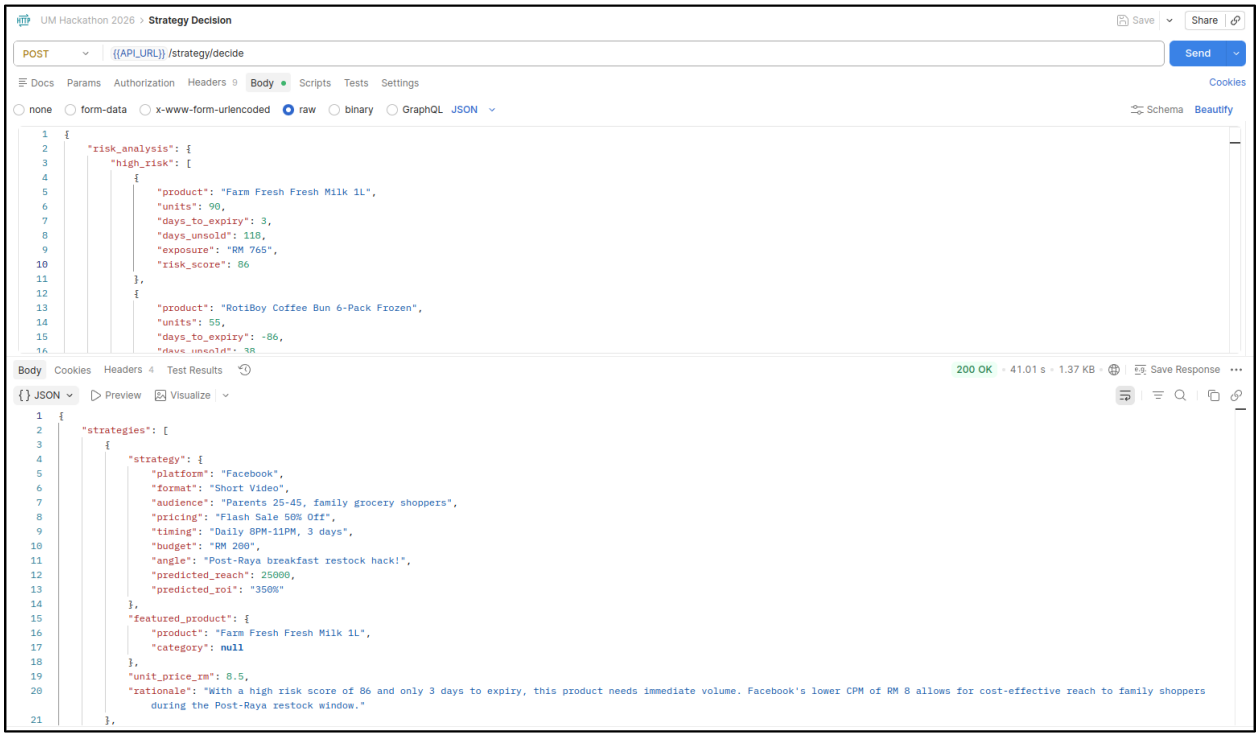

Figure 2.2.3: Postman Result (Strategy Decision)

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-03	POST /api/v1/strategy/decide → GLM (strategy)	Send risk_analysis + live_context JSON (from IT-02 + IT-04 or minimal valid bodies).	Should return HTTP 200 with strategies (≥1), area, generated_at.	{ "strategies": [{ "strategy": { "platform": "Facebook", "format": "Short Video", "audience": "Parents 25-45, family grocery shoppers", "pricing": "Flash Sale 50% Off", "timing": "Daily 8PM-11PM, 3 days", "budget": "RM 200", "angle": "Post-Raya breakfast restock hack!", "predicted_reach": 25000, "predicted_roi": "350%" }, "featured_product": { "product": "Farm Fresh Fresh Milk 1L", "category": null },	Passed

				"unit_price_rm": 8.5,
				"rationale": "With a high risk score of 86 and only 3 days to expiry, this product needs immediate volume. Facebook's lower CPM of RM 8 allows for cost-effective reach to family shoppers during the Post-Raya restock window."
				},
				{
				"strategy": {
				"platform": "TikTok",
				"format": "Reel",
				"audience": "Young adults 18-35, foodies and snack lovers",
				"pricing": "Bundle Buy 2 Free 1",
				"timing": "7PM-10PM, 5 days",
				"budget": "RM 300",
				"angle": "Vesak snack hack ready in minutes!",
				
				"predicted_reach": 25000,
				"predicted_roi": "280%"	

				},
				
				"featured_product": {
				"product": "RotiBoy Coffee Bun 6-Pack Frozen",
				"category": null
				},
				"unit_price_rm": 12.91,
				"rationale": "This high-risk item (risk 76) requires urgent clearance, and TikTok's quick recipe reel trend perfectly suits a frozen snack. Targeting the 7PM-10PM peak leverages late-night cravings ahead of Vesak Day."
				}
				],
				"area": "Malaysia",
				"generated_at": "2026-05-02T06:34:58Z"
				}	

Email: umhackathon@um.edu.my

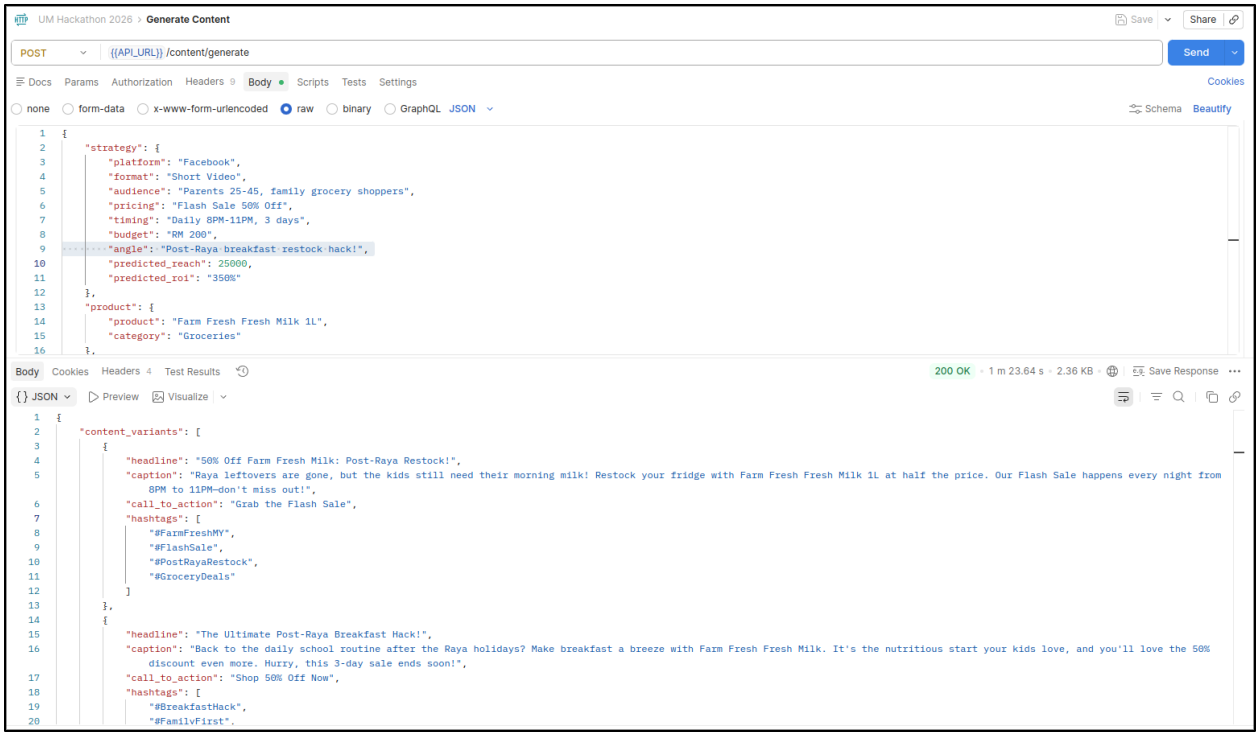

Figure 2.2.4: Postman Result (Generate Content)

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-04	POST /api/v1/content/generate → Z.A.I GLM text + GLM-Image	Send JSON with a valid strategy (AdStrategy fields), optional product	Should return HTTP 200. Body includes content_variants (copy), image	{ "content_variants": [{ "headline": "50% Off Farm Fresh Milk: Post-Raya Restock!", "caption": "Raya leftovers are gone, but the kids still need their morning milk! Restock your fridge with Farm Fresh Fresh Milk 1L at half the price. Our Flash Sale happens every night from 8PM to 11PM—don't miss out!", "call_to_action": "Grab the Flash Sale", "hashtags": ["#FarmFreshMY",	Passed

		(Content Product), optional area. Server has ZAI_API_KEY (text + image).	(url/base64/mime), image_prompt, generated_at.	"#FlashSale", "#PostRayaRestock", "#GroceryDeals"] }, { "headline": "The Ultimate Post-Raya Breakfast Hack!", "caption": "Back to the daily school routine after the Raya holidays? Make breakfast a breeze with Farm Fresh Fresh Milk. It's the nutritious start your kids love, and you'll love the 50% discount even more. Hurry, this 3-day sale ends soon!", "call_to_action": "Shop 50% Off Now", "hashtags": ["#BreakfastHack", "#FamilyFirst", "#FarmFreshMilk", "#RayaHacks"] }, { "headline": "3 Nights Only: Half-Price Milk Restock!", "caption": "Your Raya guests have left, and the fridge is empty. Time for a quick restock hack! Get Farm Fresh Fresh Milk 1L at 50% OFF during our nightly Flash Sale from 8PM to 11PM. Only 3 days left to save big on your grocery essentials!", "call_to_action": "Claim Your Deal", "hashtags": ["#NightSale", "#FarmFresh1L", "#HalfPrice", "#SmartShopper"	

				]
				}
				],
				"image_prompt": "A photorealistic commercial shot of a Farm Fresh Fresh Milk 1L bottle sitting on a bright, clean wooden breakfast table, splashing fresh white milk around its base, accompanied by a plate of toasted bread and kaya. The hero product features the distinctive Farm Fresh branding with its signature blue and white label, clearly showing the 1L size. Hovering dynamically over the product is a vibrant, eye-catching 3D discount badge that reads '50% OFF' in bold yellow text on a red background, with a smaller tag below it stating '8PM-11PM'. The lighting is bright, natural morning sunlight streaming in, creating a fresh, wholesome, and inviting mood. The color palette emphasizes crisp whites, vibrant blues from the milk bottle, and the warm tones of the breakfast spread, perfectly conveying a post-Raya breakfast restock hack.",
				"image": {
				"mime_type": null,
				"base64": null,
				"url": "https://mfile.z.ai/1777704184777-2bb7a9456af9456a9d64c38b71395efd.png?ufileattname=20260502144221eaaf4546b81d4636_watermark.png"
				},
				"generated_at": "2026-05-02T06:43:04Z"
				}	

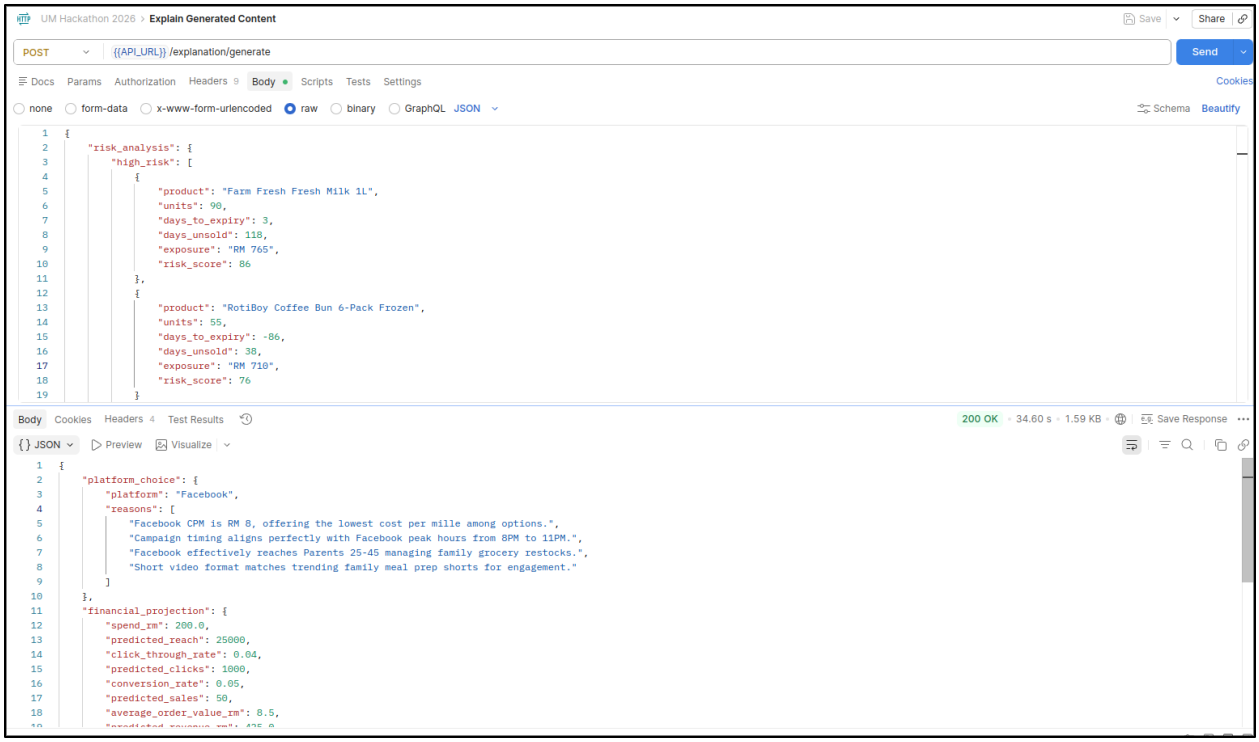

Figure 2.2.5: Postman Result (Explanation Generation)

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-05	POST /api/v1/explanation/generate → Z.A.I GLM + deterministic finance	Send JSON with risk_analysis, live_context, strategy; optional content_variants, product, unit_price_rm, area.	Should have HTTP 200. Body includes platform_choice, financial_projection, risk_vs_reward, area, generated_at.	{ "platform_choice": { "platform": "Facebook", "reasons": ["Facebook CPM is RM 8, offering the lowest cost per mille among options.", "Campaign timing aligns perfectly with Facebook peak	Passed

				hours from 8PM to 11PM.", "Facebook effectively reaches Parents 25-45 managing family grocery restocks.", "Short video format matches trending family meal prep shorts for engagement."] }, "financial_projection": { "spend_rm": 200.0, "predicted_reach": 25000, "click_through_rate": 0.04, "predicted_clicks": 1000, "conversion_rate": 0.05, "predicted_sales": 50, "average_order_value_rm": 8.5, "predicted_revenue_rm": 425.0, "roi_percent": 112.5,	

				"summary_line": "Spend RM 200 → Reach 25,000 → 1,000 clicks → 50 sales → RM 425 revenue (ROI 112%)" }, "risk_vs_reward": { "risks": ["Only 3 days to expiry creates extreme fulfillment urgency and potential spoilage.", "Flash Sale 50% Off eliminates margin on high exposure inventory.", "Post-Raya audience might be fatigued from heavy holiday spending and ads.", "90 units is low stock; high reach could cause fast stockouts."], "rewards": ["50% Off creates strong scarcity leverage to clear 3-day expiry stock instantly.", "Post-Raya angle targets families restocking daily essentials after holiday	

				depletion.", "Short video hack format fits trending recipe reels for high shareability.", "Clearing high risk inventory prevents total loss on remaining exposure."], "verdict": "Cautious go due to extreme expiry urgency requiring flawless logistics." }, "area": "Malaysia", "generated_at": "2026-05-02T08:20:51Z" }	

3. Defect Log & Fix Traceability (Root Cause Analysis)

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
DEF-01	Multi-Format Inventory Upload & Validation Failures	1. Upload an Excel file (.xlsx) with numeric data in stock_level or price. 2. Upload a file with Malaysian/UK date formats (e.g., 30/8/2026). 3. Upload a file missing expiry_date or date_added.	Type Error: 'int' object has no attribute 'strip'	The backend parser assumed all data from files were strings and tried to .strip() them, causing crashes when encountering pre-typed numeric or date objects from Excel.	https://github.com/tzee27/AdsGenerator/commit/30059348f811012557d44cbed98fac6c36149426
DEF-02	"Regenerate" for ad copy could rewrite more than the sections the user selected (partial regeneration not enforced end-to-end).	1. Generate final content. 2. Open regenerate UI and select only one section (e.g. hashtags). 3. Run regeneration observe unrelated fields also changing vs. staying stable.	(User-visible: wrong copy/hashtags vs. expectation; backend depended on model returning full content_variants.)	Regeneration prompt/merge path treated the model output as a full replacement variant set instead of patching only requested fields onto the current variant.	https://github.com/tzee27/AdsGenerator/commit/86c36676ff90e1cd7a3a110cef06d585bc7cae76

Email: umhackathon@um.edu.my

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)	Fix Commit Hash / PR Link
DEF-03	After deploying the Vite frontend (e.g. Vercel) against a hosted API (e.g. Render), strategies / finalize and other API calls failed: either CORS blocked the browser from reading responses, or POST requests behaved like GET and returned 405 on POST-only routes	1. Set VITE_API_URL to the HTTP URL of the deployed API (non-local). 2. Open the deployed SPA and sign in. 3. Upload inventory and trigger Phase A or Phase B. 4. Observe network tab: CORS errors and/or 405 on what should be a POST. 5. (Variant) Use a Vercel preview hostname not listed in CORS_ALLOW_ORIGINS — preflight fails.	Browser console: CORS / "blocked by CORS policy"; or 405 Method Not Allowed after redirect (POST upgraded incorrectly).	1. Static allowlist CORS did not cover changing Vercel preview hostnames. 2. Hosting providers often redirect HTTP → HTTPS; following that with GET can break POST multipart/JSON calls from the SPA when the client used http:// for the API base.	https://github.com/tzee27/AdsGenerator/commit/49c94cf496713569136fc23e672c60473fc90590

4. Known Issues & Deferred Technical Debt

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral	GitHub Issue Link
DEFER-001	**Single-Sheet Excel Limitation:** The system only reads the first worksheet in an Excel file. If users store inventory on Sheet 2 or Sheet 3, that data is ignored.	Low priority for the hackathon; most users keep primary data on the first sheet.	https://github.com/tzee27/AdsGenerator/commit/30059348f811012557d44cbed98fac6c36149426
DEFER-002	**Duplicate Product Detection:** The validator doesn't alert the user if the same product name appears multiple times in one file, which could lead to redundant AI analysis.	Requires cross-referencing all rows, which would slow down the initial upload speed for large files..	https://github.com/tzee27/AdsGenerator/commit/889b530f513f60939cc543099a83a152f46754d4
DEFER-003	**UI Blocking on Massive Files:** For extremely large files (e.g., 50,000+ rows), the validation loop runs on the main thread, which might cause minor UI lag.	Current inventory files are small; implementing a Web Worker for multi-threading is a "nice-to-have" for future scaling.	https://github.com/tzee27/AdsGenerator/commit/baef19702d677a78b84a5e03683e4b767b3c6e1a

5. Live Demo / UAT Risks

- **Risk 1:** Inventory uploads are capped at 5 MB and must be valid CSV or Excel. The strategies endpoint rejects larger files with HTTP 413 (MAX_UPLOAD_BYTES = 5 * 1024 * 1024 in backend/app/api/v1/endpoints/ads.py). Only .csv, .xlsx, and .xls are accepted there. For Excel, the UI parses only the first worksheet (see workbook.SheetNames[0] in frontend/src/pages/ContentGeneration.jsx). Judges should use a small-to-medium file under 5 MB with the required columns (product_name, category, stock_level, price per your risk analyser).

- **Risk 2:** Phase A (strategies) can take on the order of minutes, not seconds, because live market context uses web search and the backend chat timeout is 180 seconds

(ZAI_CHAT_TIMEOUT_SECONDS in backend/app/core/config.py, with an inline note that web search often needs 60–90s). Phase B (finalize) adds image generation with a 90s image timeout and HD-quality images that the config comments describe as roughly ~20s (vs standard ~5–10s). Ask judges to start one run, wait for the response, and avoid double-submitting; duplicate requests increase load on Z.AI and can surface as 502/429-style failures.

- **Risk 3:** The ads API requires a signed-in user (Firebase) — strategies, finalize, and regenerate-sections all use Depends(get_current_user) in ads.py. Demo accounts must be able to sign in, and the deployed frontend's origin must be allowed by backend CORS (CORS_ALLOW_ORIGINS / optional CORS_ALLOW_ORIGIN_REGEX in config.py); a mismatch looks like "API broken" in the browser. Outcomes also depend on Z.AI (keys present, quotas, network): missing configuration yields 503; upstream errors map to 502 per the same file's docstring.